

Решения задач по min и max

Python • 6–8 классы • Материал для преподавателя

Переменные и цикл while

Здесь собраны решения девяти задач урока и домашнего задания Д1. Для каждой задачи приведены условие, идея алгоритма, рабочий код, тесты и типичная ошибка. Готовые решения открывайте после самостоятельной попытки ученика.

Правила ввода и ограничения

Каждый `input()` читает одно целое число с отдельной строки. В таблицах тестов числа записаны через запятую для компактности; при запуске вводите их по строкам. Приглашения к вводу в коде отсутствуют.

№ 1 читает ровно три числа, включая возможный ноль. В № 2–9 и Д1 первый 0 завершает поток и не является данными. Непустота гарантирована условием; в № 7–8 гарантированы минимум два числа. Натуральные числа здесь положительные.

Код рассчитан на корректный ввод по условию. Первый 0 в задачах о непустом потоке находится вне условия: у пустого набора нет минимума и максимума. Его нельзя засчитывать как корректный тест с ответом 0 0. Для отдельной версии с проверкой ввода сначала обработайте `x == 0` сообщением «Нет данных», а поиск выполняйте только в другой ветке.

Как засчитать основной маршрут

Цель: каждый самостоятельно решает № 1–4. Для № 4 проверьте все шесть тестов на странице 5, начальное значение из первого элемента, исключение сигнала 0, один проход и порядок вывода. Затем попросите объяснить, что хранят `mn` и `mx` после очередного шага.

Навигация

Материал	Страницы
№ 1–4 · основной маршрут	2–5
№ 5–7 · дальнейшее продвижение	6–8
№ 8–9 · буфер	9–10
Д1 · домашнее задание	11

Все алгоритмы используют постоянное число переменных, без списков и встроенных `min()`, `max()`, `sorted()`.

Потоковые решения читают данные один раз: время $O(n)$, дополнительная память $O(1)$.

Основа: «План_урока_Полищук_MIN-MAX.pdf», с. 1–4; [knowledge-base/baza_znaniy_prepodavatelya.md](#); [knowledge-base/podacha_teorii.md](#). Условия и шкала взяты из плана; пояснения, решения и дополнительные тесты подготовлены к уроку.

1 Минимум из трёх

Введите три целых числа с разных строк. Выведите наименьшее. Обновляйте переменную `mn` последовательными сравнениями; цикл пока не обязателен.

Идея решения

Первое число уже является минимумом среди одного прочитанного числа. После каждого сравнения `mn` хранит наименьшее из уже прочитанных чисел. Ноль здесь обычное число: сигнала завершения в задаче № 1 нет.

```
mn = int(input())
x = int(input())
if x < mn:
    mn = x
x = int(input())
if x < mn:
    mn = x
print(mn)
```

Проверка

Ввод по одному числу на строке	Ожидаемый вывод
-4, -9, -2	-9
5, 5, 5	5
0, 8, -1	-1

Частая ошибка

Три независимых сравнения с исходным первым числом могут потерять промежуточный рекорд. Сравнивайте с обновлённым `mn`.

2 Максимум потока до нуля

Введите непустую последовательность целых чисел до 0. Выведите наибольшее число. Завершающий 0 не участвует в поиске.

Идея решения

mx начинается с реального элемента потока. Цикл обрабатывает остальные числа, только пока x не равен 0. Если все числа отрицательные, максимум всё равно будет одним из них.

```
mx = int(input())
x = int(input())
while x != 0:
    if x > mx:
        mx = x
    x = int(input())
print(mx)
```

Проверка

Ввод по одному числу на строке	Ожидаемый вывод
-8, -3, -5, 0	-3
6, 0	6
2, 9, 9, 4, 0	9

Частая ошибка

При mx = 0 поток -8, -3, -5 дал бы ошибочный ответ 0. Не придумывайте начальный рекорд.

3 Минимум потока до нуля

Введите непустую последовательность целых чисел до 0. Выведите наименьшее число, исключив завершающий 0.

Идея решения

После первого ввода `mn` равен первому элементу. Условие `x < mn` заменяет минимум только меньшим числом. Проверка `x != 0` выполняется до обработки очередного числа.

```
mn = int(input())
x = int(input())
while x != 0:
    if x < mn:
        mn = x
    x = int(input())
print(mn)
```

Проверка

Ввод по одному числу на строке	Ожидаемый вывод
8, 3, 5, 0	3
-8, -3, -5, 0	-8
4, 4, 0	4

Частая ошибка

Если учитывать завершающий ноль, у положительного потока 8, 3, 5 получится ложный минимум 0.

4 Два рекорда за один проход

Непустая последовательность целых чисел заканчивается 0. Найдите максимум и минимум за один проход. Выведите максимум, затем минимум через пробел. Ноль не учитывать.

Идея решения

Первый элемент задаёт оба рекорда. Каждое следующее ненулевое число сравнивается с `mx` и `mn`. После обработки любого префикса `mx` и `mn` являются его максимумом и минимумом. Поэтому после сигнала остаётся вывести итоговые рекорды.

```
x = int(input())
mn = x
mx = x
while x != 0:
    if x > mx:
        mx = x
    if x < mn:
        mn = x
    x = int(input())
print(mx, mn)
```

Проверка

Ввод по одному числу на строке	Ожидаемый вывод
7, -2, 9, 0	9 -2
-8, -3, -5, 0	-3 -8
8, 3, 5, 0	8 3
6, 0	6 6
4, 4, 4, 0	4 4
9, 2, -6, 0	9 -6

Частая ошибка

Не перепутайте порядок вывода: сначала `mx`, затем `mn`. `print` внутри `while` выведет промежуточные ответы вместо одного итогового.

5 Первое место максимума

Натуральные числа поступают до 0. Поток непустой. Выведите индекс первого наибольшего элемента; индексация с 0. Индекс храните в переменной, список не нужен.

Идея решения

Первый элемент имеет индекс 0. Для каждого следующего числа i увеличивается на 1. При $x > mx$ сохраняются и новый рекорд, и его индекс. Равное значение не меняет pos , поэтому он остаётся первым.

```
mx = int(input())
i = 0
pos = 0
x = int(input())
while x != 0:
    i = i + 1
    if x > mx:
        mx = x
        pos = i
    x = int(input())
print(pos)
```

Проверка

Ввод по одному числу на строке	Ожидаемый вывод
2, 7, 7, 3, 0	1
9, 2, 9, 0	0
5, 0	0

Частая ошибка

При $x \geq mx$ будет сохранена последняя позиция максимума вместо первой.

6 Сколько раз достигнут максимум

Введите непустой поток натуральных чисел до 0. Выведите, сколько введенных чисел равно итоговому максимуму.

Идея решения

cnt считает вхождения текущего mx. При большем числе старый счётчик больше не нужен: новый максимум пока встретился один раз. Ветка elif исключает двойной подсчёт только что найденного рекорда.

```
mx = int(input())
cnt = 1
x = int(input())
while x != 0:
    if x > mx:
        mx = x
        cnt = 1
    elif x == mx:
        cnt = cnt + 1
    x = int(input())
print(cnt)
```

Проверка

Ввод по одному числу на строке	Ожидаемый вывод
3, 3, 5, 5, 0	2
3, 3, 5, 0	1
7, 7, 7, 0	3
2, 0	1

Частая ошибка

Два отдельных if с проверками $x > mx$ и $x == mx$ могут посчитать новый рекорд дважды: после обновления mx равен x .

7 Второй по величине

Введите различные натуральные числа до 0; до сигнала есть хотя бы два числа. Выведите второе по величине.

Идея решения

После первых двух вводов $mx1$ больше $mx2$. При $x > mx1$ сначала переносим прежний $mx1$ в $mx2$, затем записываем x в $mx1$. Если x не лидер, но больше $mx2$, меняется только второе место. Различность чисел гарантирована условием.

```
mx1 = int(input())
mx2 = int(input())
if mx2 > mx1:
    temp = mx1
    mx1 = mx2
    mx2 = temp
x = int(input())
while x != 0:
    if x > mx1:
        mx2 = mx1
        mx1 = x
    elif x > mx2:
        mx2 = x
    x = int(input())
print(mx2)
```

Проверка

Ввод по одному числу на строке	Ожидаемый вывод
4, 9, 6, 0	6
2, 8, 0	2
9, 6, 4, 0	6
1, 2, 3, 4, 0	3

Частая ошибка

Если сначала выполнить $mx1 = x$, а потом $mx2 = mx1$, прежний лидер потеряется и оба рекорда станут равны x .

8 Второй минимум с повторами

До 0 поступают минимум два натуральных числа, каждое не больше 10^9 . Мысленно удалите одно вхождение наименьшего и выведите минимум оставшихся. Повторы разрешены.

Идея решения

Инициализация первыми двумя числами не требует искусственно большого значения. Пара `mn1`, `mn2` хранит два наименьших вхождения. Равный `mn1` элемент может попасть в `mn2` через `elif`. Поэтому для 1, 2, 1 ответ равен 1, а не 2.

```
mn1 = int(input())
mn2 = int(input())
if mn2 < mn1:
    temp = mn1
    mn1 = mn2
    mn2 = temp
x = int(input())
while x != 0:
    if x < mn1:
        mn2 = mn1
        mn1 = x
    elif x < mn2:
        mn2 = x
    x = int(input())
print(mn2)
```

Проверка

Ввод по одному числу на строке	Ожидаемый вывод
1, 2, 1, 0	1
5, 5, 0	5
9, 4, 7, 0	7
1000000000, 999999999, 0	1000000000

Частая ошибка

Не отбрасывайте повтор минимального значения: удаляется ровно одно вхождение, а не все минимумы.

9 Расстояние между первыми рекордами

В непустом потоке целых ненулевых чисел до 0 найдите расстояние между позициями первого минимума и первого максимума: модуль разности индексов, не разность значений.

Идея решения

При строгом улучшении обновляются значение и его позиция. Равенство сохраняет первую позицию. Разность `pos_mx - pos_mn` может быть отрицательной, поэтому меняем её знак при необходимости. Если все числа равны, обе первые позиции равны 0.

```
x = int(input())
mn = x
mx = x
pos_mn = 0
pos_mx = 0
i = 0
x = int(input())
while x != 0:
    i = i + 1
    if x < mn:
        mn = x
        pos_mn = i
    if x > mx:
        mx = x
        pos_mx = i
    x = int(input())
distance = pos_mx - pos_mn
if distance < 0:
    distance = -distance
print(distance)
```

Проверка

Ввод по одному числу на строке	Ожидаемый вывод
-1, -2, 1, 7, 2, 0	2
7, -2, 7, -2, 0	1
4, 4, 0	0
6, 0	0

Частая ошибка

Разность `mx - mn` измеряет размах значений. Здесь требуются позиции, поэтому вычислять нужно расстояние между `pos_mx` и `pos_mn`.

Д1 Размах потока

Для непустого потока целых чисел до 0 выведите размах $mx - mn$. Используйте переменные и `while`. Сдайте файл `.py` и три теста.

Идея решения

Размах равен разности максимального и минимального значений. Он неотрицателен. Если число одно или все числа равны, размах равен 0. Поиск рекордов совпадает с № 4, меняется только итоговый вывод.

```
x = int(input())
mn = x
mx = x
x = int(input())
while x != 0:
    if x > mx:
        mx = x
    if x < mn:
        mn = x
    x = int(input())
print(mx - mn)
```

Проверка

Ввод по одному числу на строке	Ожидаемый вывод
7, -2, 9, 0	11
-8, -3, -5, 0	5
6, 0	0

Частая ошибка

Не путайте размах с расстоянием между позициями из № 9.